

Lab 05: Design Pattern

Scenario 1: Modern library

Concept: Strategy Pattern for enhancing the capabilities of Library Items:

- MembershipStrategy - MembershipStrategy Interface, defines the contract for different membership types.
- BasicMembershipStrategy - Standard membership for regular members.
- StudentMembershipStrategy - Special membership for students with educational discounts.
- PremiumMembershipStrategy - VIP membership with maximum benefits.
- LibrarySystem – (Singleton Pattern) Ensures there is only ONE instance of LibrarySystem throughout the application.

Instructions:

- Reuse classes from Lab4 Scenario 1
- Add a New MembershipStrategy
 - Create a new membership strategy named BasicMembershipStrategy, which allows:
 - A borrowing limit of 1 items
 - A loan period of 14 days
 - A 0% discount on late fees
 - Membership costs 0
 - Create a new membership strategy named StudentMembershipStrategy, which allows:
 - A borrowing limit of 5 items
 - A loan period of 21 days
 - A 20% discount on late fees
 - Membership costs 49.0
 - Create a new membership strategy named PremiumMembershipStrategy, which allows:
 - Unlimited borrowing
 - A loan period of 30 days
 - A 0% discount on late fees
 - Membership costs 299.0

- Then update the LibraryManagementApp to:
 - Create one Basic member
 - Demonstrate one borrowing transaction
 - Calculate the late fee using baseFee = 100
- Success Criteria:
 - A BasicMembershipStrategy class exists and implements MembershipStrategy
 - A Basic member is successfully created and displayed correctly
 - The borrowing transaction shows Loan Period = 21 days
 - Late fee calculation correctly applies the discount:
 - Base fee 100 → Final fee 80 (after 20% discount)

MembershipStrategy

```
package com.library.lab05;

/**
 * STRATEGY PATTERN - MembershipStrategy Interface
 *
 * Defines the contract for different membership types.
 * Each membership type has different behaviors:
 * - Borrow limits
 * - Loan periods
 * - Late fee discounts
 *
 * Benefits of Strategy Pattern:
 * - Easy to add new membership types without modifying existing code
 * - Member can change strategy at runtime (upgrade membership)
 * - Eliminates if-else or switch statements for member types
 * - Each strategy is a separate class (Single Responsibility Principle)
 *
 * Similar to SimUDuck's FlyBehavior and QuackBehavior!
 */
public interface MembershipStrategy {
    int getBorrowLimit();
    int getLoanPeriodDays();
    double applyLateFeeDiscount(double baseFee);
    String getMembershipType();
    double getMembershipCost();
    boolean hasUnlimitedBorrowing();
}
```

BasicMembershipStrategy

```
package com.library.lab05;

/**
 * STRATEGY PATTERN - BasicMembershipStrategy
 *
 * Standard membership for regular members.
 *
 * Benefits:
 * - Borrow Limit: 1 items
 * - Loan Period: 14 days
 * - Late Fee Discount: None (0%)
 * - Cost: FREE
 *
 * This is the default strategy for all new members.
 */
public class BasicMembershipStrategy implements MembershipStrategy {

    private static final int BORROW_LIMIT = 1;
    private static final int LOAN_PERIOD = 14;
    private static final double MEMBERSHIP_COST = 0.0; // Free

    @Override
    public int getBorrowLimit() {
        return BORROW_LIMIT;
    }

    @Override
    public int getLoanPeriodDays() {
        return LOAN_PERIOD;
    }

    @Override
    public double applyLateFeeDiscount(double baseFee) {
        // No discount for basic members
        return baseFee;
    }

    @Override
    public String getMembershipType() {
        return "Basic Member";
    }

    @Override
```

```
public double getMembershipCost() {  
    return MEMBERSHIP_COST;  
}  
  
@Override  
public boolean hasUnlimitedBorrowing() {  
    return false;  
}  
}
```

StudentMembershipStrategy

```
package com.library.lab05;

/**
 * STRATEGY PATTERN - StudentMembershipStrategy
 *
 * Special membership for students with educational discounts.
 *
 * Benefits:
 * - Borrow Limit: 5 items (more for studying)
 * - Loan Period: 21 days (3 weeks for assignments)
 * - Late Fee Discount: 20% (student discount)
 * - Cost: 49 Baht/month
 *
 * Requires student ID verification.
 */
public class StudentMembershipStrategy implements MembershipStrategy {

    private static final int BORROW_LIMIT = 5;
    private static final int LOAN_PERIOD = 21;
    private static final double LATE_FEE_DISCOUNT = 0.20; // 20% discount
    private static final double MEMBERSHIP_COST = 49.0;

    @Override
    public int getBorrowLimit() {
        return BORROW_LIMIT;
    }

    @Override
    public int getLoanPeriodDays() {
        return LOAN_PERIOD;
    }

    @Override
    public double applyLateFeeDiscount(double baseFee) {
        double discount = baseFee * LATE_FEE_DISCOUNT;
        double finalFee = baseFee - discount;

        System.out.println("🎓 [Student Discount Applied]");
        System.out.println("Original Fee: " + baseFee + " Baht");
        System.out.println("Student Discount (20%): -" + discount + " Baht");
        System.out.println("Final Fee: " + finalFee + " Baht");

        return finalFee;
    }
}
```

```

    }

    @Override
    public String getMembershipType() {
        return "Student Member";
    }

    @Override
    public double getMembershipCost() {
        return MEMBERSHIP_COST;
    }

    @Override
    public boolean hasUnlimitedBorrowing() {
        return false;
    }
}

```

PremiumMembershipStrategy

```

package com.library.lab05;

/**
 * STRATEGY PATTERN - PremiumMembershipStrategy
 *
 * VIP membership with maximum benefits.
 *
 * Benefits:
 * - Borrow Limit: UNLIMITED
 * - Loan Period: 30 days
 * - Late Fee Discount: 100% (FREE - no late fees!)
 * - Cost: 299 Baht/month
 * - Priority holds and reservations
 *
 * Best value for avid readers!
 */
public class PremiumMembershipStrategy implements MembershipStrategy {

    private static final int BORROW_LIMIT = Integer.MAX_VALUE; // Unlimited
    private static final int LOAN_PERIOD = 30;
    private static final double LATE_FEE_DISCOUNT = 1.0; // 100% discount (FREE)
    private static final double MEMBERSHIP_COST = 299.0;
}

```

```

@Override
public int getBorrowLimit() {
    return BORROW_LIMIT;
}

@Override
public int getLoanPeriodDays() {
    return LOAN_PERIOD;
}

@Override
public double applyLateFeeDiscount(double baseFee) {
    double discount = baseFee * LATE_FEE_DISCOUNT;
    double finalFee = baseFee - discount;

    System.out.println("    👑 [Premium Member Privilege]");
    System.out.println("    Original Fee: " + baseFee + " Baht");
    System.out.println("    Premium Discount (100%): -" + baseFee + " Baht");
    System.out.println("    Final Fee: " + finalFee + " Baht (FREE!)");

    return 0.0; // No late fees for premium members!
}

@Override
public String getMembershipType() {
    return "Premium Member";
}

@Override
public double getMembershipCost() {
    return MEMBERSHIP_COST;
}

@Override
public boolean hasUnlimitedBorrowing() {
    return true;
}
}

```

LibrarySystem

```
public class LibrarySystem {

    // Singleton instance
    private static LibrarySystem instance;

    // Library data
    private List<LibraryItem> allItems;
    private List<Member> allMembers;
    private String libraryName;
    private String libraryLocation;

    /**
     * Private constructor - Prevents direct instantiation
     * Only getInstance() can create the instance
     */
    private LibrarySystem() {
        this.allItems = new ArrayList<>();
        this.allMembers = new ArrayList<>();
        this.libraryName = "City Central Library";
        this.libraryLocation = "123 Main Street, Downtown";
    }

    /**
     * SINGLETON METHOD: Thread-safe getInstance()
     * Returns the single instance of LibrarySystem
     * Creates instance on first call, returns same instance on subsequent calls
     */
    public static synchronized LibrarySystem getInstance() {
        if (instance == null) {
            instance = new LibrarySystem();
            System.out.println("[Singleton] LibrarySystem initialized (First time)");
        }
        return instance;
    }

    // Getters
    public String getLibraryName() {
        return libraryName;
    }

    public String getLibraryLocation() {
        return libraryLocation;
    }
}
```



```

}

public List<LibraryItem> getAllItems() {
    return new ArrayList<>(allItems);
}

public List<Member> getAllMembers() {
    return new ArrayList<>(allMembers);
}

// Library Operations
public void addItem(LibraryItem item) {
    allItems.add(item);
    System.out.println(" ☒ Item added to system: " + item.getTitle());
}

public void addMember(Member member) {
    allMembers.add(member);
    System.out.println(" ☒ Member registered: " + member.getName());
}

public int getTotalItems() {
    return allItems.size();
}

public int getTotalMembers() {
    return allMembers.size();
}

public int getAvailableItemsCount() {
    return (int) allItems.stream().filter(LibraryItem::isAvailable).count();
}

public int getBorrowedItemsCount() {
    return (int) allItems.stream().filter(item -> !item.isAvailable()).count();
}

/**
 * Display library statistics
 */
public void displayStatistics() {
    System.out.println("\n" + "=".repeat(60));
    System.out.println("  LIBRARY SYSTEM STATISTICS (Singleton Instance)");
    System.out.println("=".repeat(60));
    System.out.println("\nLibrary: " + libraryName);
}

```

```

        System.out.println("Location: " + libraryLocation);
        System.out.println("\n📊 STATISTICS:");
        System.out.println("  Total Items: " + getTotalItems());
        System.out.println("  Available Items: " + getAvailableItemsCount());
        System.out.println("  Borrowed Items: " + getBorrowedItemsCount());
        System.out.println("  Total Members: " + getTotalMembers());
        System.out.println("\n💡 Note: This is the ONLY instance of LibrarySystem in
the application!");
        System.out.println("  (Singleton Pattern ensures single centralized
management)");
    }

    /**
     * Find item by title
     */
    public LibraryItem findItemByTitle(String title) {
        return allItems.stream()
            .filter(item -> item.getTitle().equalsIgnoreCase(title))
            .findFirst()
            .orElse(null);
    }

    /**
     * Find member by ID
     */
    public Member findMemberById(String memberId) {
        return allMembers.stream()
            .filter(member -> member.getMemberId().equals(memberId))
            .findFirst()
            .orElse(null);
    }
}

```

LibraryManagementApp

```
package com.library.lab05;

public class LibraryManagementApp {

    public static void main(String[] args) {
        // ===== SINGLETON PATTERN DEMO =====
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  DESIGN PATTERNS: SINGLETON & STRATEGY");
        System.out.println("=".repeat(70));

        // Get the single instance of LibrarySystem (Singleton Pattern)
        System.out.println("\n[STEP 1] Initializing LibrarySystem (Singleton Pattern:");
        LibrarySystem librarySystem = LibrarySystem.getInstance();

        // Try to get instance again - should return same instance
        System.out.println("Getting LibrarySystem instance again:");
        LibrarySystem sameInstance = LibrarySystem.getInstance();
        System.out.println("  ☑ Same instance? " + (librarySystem == sameInstance));

        // ===== CREATE LIBRARY ITEMS =====
        System.out.println("\n[STEP 2] Creating Library Items:");

        // Add Physical Books
        PhysicalBook book1 = new PhysicalBook("Java Programming", "John Smith", "978-0134685991", 450.0, "A1-04");
        PhysicalBook book2 = new PhysicalBook("Clean Code", "Robert Martin", "978-0132350884", 520.0, "B2-15");
        PhysicalBook book3 = new PhysicalBook("Design Patterns", "Gang of Four", "978-0201633612", 680.0, "A3-22");
        PhysicalBook book4 = new PhysicalBook("The Art of War", "Sun Tzu", "978-1599869773", 250.0, "C1-10");
        PhysicalBook book5 = new PhysicalBook("Refactoring", "Martin Fowler", "978-0134757599", 550.0, "A2-08");
        librarySystem.addItem(book1);
        librarySystem.addItem(book2);
        librarySystem.addItem(book3);
        librarySystem.addItem(book4);
        librarySystem.addItem(book5);
    }
}
```

```

// Add E-Books
EBook ebook1 = new EBook("Effective Java", "Joshua Bloch", "978-0134685991",
    "https://library.ebooks.com/effective-java.pdf", 5.2);
EBook ebook2 = new EBook("Python Crash Course", "Eric Matthes", "978-
1593279288",
    "https://library.ebooks.com/python-crash.pdf", 8.7);
librarySystem.addItem(ebook1);
librarySystem.addItem(ebook2);

// ===== STRATEGY PATTERN: CREATE MEMBERS WITH DIFFERENT
// STRATEGIES =====
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 3] STRATEGY PATTERN: Creating Members with
Different Strategies");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Each member has a
MembershipStrategy!");
System.out.println(" Strategy determines: borrow limit, loan period, late
fee discount\n");

// Create members with different strategies
Member basicMember = new Member("M001", "Somsak", new
BasicMembershipStrategy());
Member studentMember = new Member("M002", "Suda", new
StudentMembershipStrategy());
Member premiumMember = new Member("M003", "Somchai", new
PremiumMembershipStrategy());

librarySystem.addMember(basicMember);
librarySystem.addMember(studentMember);
librarySystem.addMember(premiumMember);

// Display member information
System.out.println("--- MEMBER INFORMATION ---");
for (Member member : librarySystem.getAllMembers()) {
    member.displayMemberInfo();
}

// ===== STRATEGY PATTERN: TESTING DIFFERENT BORROW BEHAVIORS
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 4] STRATEGY PATTERN: Testing Different Borrow
Behaviors");
System.out.println("=".repeat(70));
System.out.println("\n💎 Each member type has different borrow limits and
loan periods!\n");

```

```

// Basic Member borrows (limit: 1, period: 14 days)
System.out.println("--- Basic Member (Somsak) Borrows ---");
book1.checkOut(basicMember);

// Student Member borrows (limit: 5, period: 21 days)
System.out.println("\n--- Student Member (Suda) Borrows ---");
book2.checkOut(studentMember);

// Premium Member borrows (unlimited, period: 30 days)
System.out.println("\n--- Premium Member (Somchai) Borrows ---");
book3.checkOut(premiumMember);
ebook1.checkOut(premiumMember);
ebook2.checkOut(premiumMember); // Premium can borrow unlimited!

// ===== STRATEGY PATTERN: LATE FEE COMPARISON
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 5] STRATEGY PATTERN: Late Fee Discounts");
System.out.println("=".repeat(70));
System.out.println("\n💎 Different membership types get different late fee
discounts!\n");

double baseFee = 100.0;
System.out.println("Base Late Fee: " + baseFee + " Baht\n");

System.out.println("❶ Basic Member (No discount):");
double basicFee = basicMember.calculateLateFee(baseFee);
System.out.println(" Final: " + basicFee + " Baht\n");

System.out.println("❷ Student Member (20% discount):");
double studentFee = studentMember.calculateLateFee(baseFee);
System.out.println(" Final: " + studentFee + " Baht\n");

System.out.println("❸ Premium Member (100% FREE!):");
double premiumFee = premiumMember.calculateLateFee(baseFee);
System.out.println(" Final: " + premiumFee + " Baht\n");

// ===== STRATEGY PATTERN: RUNTIME STRATEGY CHANGE
System.out.println("\n" + "=".repeat(70));
System.out.println(" [STEP 5] STRATEGY PATTERN: Changing Strategy at
Runtime");
System.out.println("=".repeat(70));
System.out.println("\n💎 KEY CONCEPT: Members can upgrade/downgrade
membership dynamically!\n");

```

```
// Basic member upgrades to Premium
System.out.println("--- Somsak (MemberID:M001) Upgrades from Basic to Premium
---");
Member somsak = librarySystem.findMemberById("M001");

// Somsak tries to borrow more than basic limit
System.out.println("\n--- Before Upgrade: Somsak can only borrow 1 item ---
");
book5.checkOut(basicMember); // Should fail (limit reached)

// Upgrade to Premium!
somsak.setMembershipStrategy(new PremiumMembershipStrategy());
somsak.displayMemberInfo();

// Now Somsak can borrow more items!
System.out.println("\n--- After Upgrade: Somsak can now borrow more! ---");
book5.checkOut(basicMember); // Now unlimited!

// ===== LIBRARY SYSTEM STATISTICS =====
librarySystem.displayStatistics();
}
}
```

Output:

```
=====
DESIGN PATTERNS: SINGLETON & STRATEGY
=====

[STEP 1] Initializing LibrarySystem (Singleton Pattern):
📁 [Singleton] LibrarySystem initialized (First time)
Getting LibrarySystem instance again:
✅ Same instance? true

[STEP 2] Creating Library Items:
✅ Item added to system: Java Programming
✅ Item added to system: Clean Code
✅ Item added to system: Design Patterns
✅ Item added to system: The Art of War
✅ Item added to system: Refactoring
✅ Item added to system: Effective Java
✅ Item added to system: Python Crash Course

=====

[STEP 3] STRATEGY PATTERN: Creating Members with Different Strategies
=====

💡 KEY CONCEPT: Each member has a MembershipStrategy!
Strategy determines: borrow limit, loan period, late fee discount

✅ Member registered: Somsak
✅ Member registered: Suda
✅ Member registered: Somchai
--- MEMBER INFORMATION ---

--- MEMBER INFORMATION ---
ID: M001
Name: Somsak
Membership Type: Basic Member
Membership Cost: 0.0 Baht
Borrow Limit: 1
Currently Borrowed: 0
Loan Period: 14 days

--- MEMBER INFORMATION ---
ID: M002
Name: Suda
Membership Type: Student Member
Membership Cost: 49.0 Baht
Borrow Limit: 5
Currently Borrowed: 0
Loan Period: 21 days

--- MEMBER INFORMATION ---
ID: M003
```

Name: Somchai
Membership Type: Premium Member
Membership Cost: 299.0 Baht
Borrow Limit: Unlimited
Currently Borrowed: 0
Loan Period: 30 days

[STEP 4] STRATEGY PATTERN: Testing Different Borrow Behaviors

✦ Each member type has different borrow limits and loan periods!

--- Basic Member (Somsak) Borrows ---

✓ Item 'Java Programming' has been checked out successfully.
📖 Borrowed by Somsak (Basic Member)
📅 Loan Period: 14 days
📅 Return Due Date: 2026-03-19

--- Student Member (Suda) Borrows ---

✓ Item 'Clean Code' has been checked out successfully.
📖 Borrowed by Suda (Student Member)
📅 Loan Period: 21 days
📅 Return Due Date: 2026-03-26

--- Premium Member (Somchai) Borrows ---

✓ Item 'Design Patterns' has been checked out successfully.
📖 Borrowed by Somchai (Premium Member)
📅 Loan Period: 30 days
📅 Return Due Date: 2026-04-04
✓ Item 'Effective Java' has been checked out successfully.
📖 Borrowed by Somchai (Premium Member)
📅 Loan Period: 30 days
📅 Return Due Date: 2026-04-04
✓ Item 'Python Crash Course' has been checked out successfully.
📖 Borrowed by Somchai (Premium Member)
📅 Loan Period: 30 days
📅 Return Due Date: 2026-04-04

[STEP 5] STRATEGY PATTERN: Late Fee Discounts

✦ Different membership types get different late fee discounts!

Base Late Fee: 100.0 Baht

1 Basic Member (No discount):
Final: 100.0 Baht

2 Student Member (20% discount):

👉 [Student Discount Applied]

Original Fee: 100.0 Baht

Student Discount (20%): -20.0 Baht

Final Fee: 80.0 Baht

Final: 80.0 Baht

3 Premium Member (100% FREE!):

👑 [Premium Member Privilege]

Original Fee: 100.0 Baht

Premium Discount (100%): -100.0 Baht

Final Fee: 0.0 Baht (FREE!)

Final: 0.0 Baht

=====

[STEP 5] STRATEGY PATTERN: Changing Strategy at Runtime

=====

💡 KEY CONCEPT: Members can upgrade/downgrade membership dynamically!

--- Somsak (MemberID:M001) Upgrades from Basic to Premium---

--- Before Upgrade: Somsak can only borrow 1 item ---

⚠️ Member Somsak has reached the borrow limit (1) for Basic Member.
Borrow request denied for member Somsak.

--- MEMBER INFORMATION ---

ID: M001

Name: Somsak

Membership Type: Premium Member

Membership Cost: 299.0 Baht

Borrow Limit: Unlimited

Currently Borrowed: 1

Loan Period: 30 days

--- After Upgrade: Somsak can now borrow more! ---

✅ Item 'Refactoring' has been checked out successfully.

📖 Borrowed by Somsak (Premium Member)

📅 Loan Period: 30 days

📅 Return Due Date: 2026-04-04

=====

📊 LIBRARY SYSTEM STATISTICS (Singleton Instance)

=====

Library: City Central Library

Location: 123 Main Street, Downtown

📊 STATISTICS:

Total Items: 7

Available Items: 1

Borrowed Items: 6
Total Members: 3